

Simulador: <https://wokwi.com/projects/458328445294940161>

Código completo:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// Definição dos Pinos
#define LDR_PIN A0
#define DHTPIN A1
#define TRIG_PIN 7
#define ECHO_PIN 8
#define LED_PIN 13

// Configuração do DHT11
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// Limites de velocidade
#define VELOCIDADE_MAX_DIA 10.0
#define VELOCIDADE_MAX_NOITE 20.0

LiquidCrystal_I2C lcd(0x27, 16, 2);

// Variáveis de Distância, Velocidade e Destino
unsigned long tempoAnterior = 0;
const long intervaloMedicao = 500;
float distanciaAnterior = 0;
float velocidadeAtual = 0;

// Variáveis do DHT11
unsigned long tempoAnteriorDHT = 0;
const long intervaloDHT = 2000;
float temperatura = 0;
float umidade = 0;

// Distância total até a próxima cidade (Começa em 5000 metros)
float metrosParaCidade = 5000.0;

// Variáveis de "Memória"
float velocidadeDisplay = 0;
unsigned long tempoUltimaInfracao = 0;
unsigned long tempoPicoVelocidade = 0;

// Controle de Telas (Agora são 5 telas na ordem solicitada)
int telaAtual = 0;
```

```

unsigned long tempoTela = 0;
const long intervaloTela = 3000;
bool limparTela = true;
bool estaDeNoite = false;

// Relógio Interno (Simulado)
unsigned long tempoRelogioAnterior = 0;
int dia = 12; int mes = 3; int hora = 12; int minuto = 0; int segundo = 0;

void setup() {
  Serial.begin(9600); // INICIA A COMUNICAÇÃO COM O SERIAL MONITOR

  pinMode(LDR_PIN, INPUT);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);

  dht.begin(); // Inicia o sensor DHT11

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);

  lcd.print("Iniciando PMV...");
  Serial.println("Iniciando PMV..."); // Envia para o Serial Monitor

  delay(2000);
}

float lerDistancia() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duracao = pulseIn(ECHO_PIN, HIGH, 30000);
  if (duracao == 0) return distanciaAnterior;
  return (duracao * 0.0343) / 2;
}

void atualizarRelogio() {
  unsigned long tempoAtual = millis();
  if (tempoAtual - tempoRelogioAnterior >= 1000) {
    tempoRelogioAnterior = tempoAtual;
    segundo++;
    if (segundo >= 60) {
      segundo = 0; minuto++;
    }
  }
}

```

```

    if (minuto >= 60) {
        minuto = 0; hora++;
        if (hora >= 24) { hora = 0; dia++; }
    }
}
// Tela 0 é a de Data/Hora, atualiza a cada segundo
if(telaAtual == 0) atualizarDisplay();
}
}

void loop() {
    unsigned long tempoAtual = millis();
    atualizarRelogio();

    // Continua lendo a luz para a regra de limite de velocidade, mesmo sem mostrar no
display
    estaDeNoite = (analogRead(LDR_PIN) > 512);
    float limiteAtual = estaDeNoite ? VELOCIDADE_MAX_NOITE : VELOCIDADE_MAX_DIA;

    // 1. LEITURA DO DHT11 (A cada 2 segundos)
    if (tempoAtual - tempoAnteriorDHT >= intervaloDHT) {
        tempoAnteriorDHT = tempoAtual;
        float t = dht.readTemperature();
        float h = dht.readHumidity();

        if (!isnan(t) && !isnan(h)) {
            temperatura = t;
            umidade = h;
        }

        // Atualiza se estiver na tela de Temp/Umidade (Tela 1)
        if(telaAtual == 1) atualizarDisplay();
    }

    // 2. CÁLCULO DE VELOCIDADE E DISTÂNCIA
    if (tempoAtual - tempoAnterior >= intervaloMedicao) {
        float distanciaAtual = lerDistancia();
        float deltaDistancia = abs(distanciaAnterior - distanciaAtual);
        float deltaTempo = intervaloMedicao / 1000.0;

        velocidadeAtual = (deltaDistancia / deltaTempo) * 3.6;

        if (deltaDistancia > 0) {
            metrosParaCidade = metrosParaCidade - deltaDistancia;
            if (metrosParaCidade < 0) metrosParaCidade = 0;
        }

        if (velocidadeAtual > 5.0) {

```

```

    velocidadeDisplay = velocidadeAtual;
    tempoPicoVelocidade = tempoAtual;
}

if (velocidadeAtual > limiteAtual) {
    digitalWrite(LED_PIN, HIGH);
    tempoUltimaInfracao = tempoAtual;
}

distanciaAnterior = distanciaAtual;
tempoAnterior = tempoAtual;

// Atualiza se estiver nas telas de sensor(2) ou cidade(3)
if(telaAtual == 2 || telaAtual == 3) atualizarDisplay();
}

// 3. TEMPORIZADORES DE DESLIGAMENTO DO LED E ZERAR VELOCIDADE
if (digitalRead(LED_PIN) == HIGH && (tempoAtual - tempoUltimaInfracao >= 3000)) {
    digitalWrite(LED_PIN, LOW);
}

if (velocidadeDisplay > 0 && (tempoAtual - tempoPicoVelocidade >= 3000)) {
    velocidadeDisplay = 0;
    if(telaAtual == 2) atualizarDisplay(); // Atualiza tela de velocidade
}

// 4. TROCA DE TELAS (Agora são 5 telas: 0 a 4)
if (tempoAtual - tempoTela >= intervaloTela) {
    tempoTela = tempoAtual;
    telaAtual = (telaAtual + 1) % 5;
    limparTela = true;
    atualizarDisplay();
}
}

void atualizarDisplay() {
    if (limparTela) {
        lcd.clear();
        Serial.println("\n--- MUDANCA DE TELA ---"); // Separador no Serial Monitor
        limparTela = false;
    }

    switch (telaAtual) {
        case 0: // 1. DATA E HORA
            // Atualiza o LCD
            lcd.setCursor(0, 0);
            lcd.print("Data: ");
            if(dia < 10) lcd.print('0'); lcd.print(dia); lcd.print('/');

```

```

if(mes < 10) lcd.print('0'); lcd.print(mes);
lcd.setCursor(0, 1);
lcd.print("Hora: ");
if(hora < 10) lcd.print('0'); lcd.print(hora); lcd.print(':');
if(minuto < 10) lcd.print('0'); lcd.print(minuto); lcd.print(':');
if(segundo < 10) lcd.print('0'); lcd.print(segundo);

// Atualiza o Serial Monitor
Serial.print("[INFO] Data: ");
if(dia < 10) Serial.print('0'); Serial.print(dia); Serial.print('/');
if(mes < 10) Serial.print('0'); Serial.print(mes);
Serial.print(" | Hora: ");
if(hora < 10) Serial.print('0'); Serial.print(hora); Serial.print(':');
if(minuto < 10) Serial.print('0'); Serial.print(minuto); Serial.print(':');
if(segundo < 10) Serial.print('0'); Serial.println(segundo);
break;

```

case 1: // 2. TEMPERATURA E UMIDADE JUNTAS

```

// Atualiza o LCD
lcd.setCursor(0, 0);
lcd.print("Temp: ");
lcd.print(temperatura, 1);
lcd.print((char)223);
lcd.print("C  ");
lcd.setCursor(0, 1);
lcd.print("Umid: ");
lcd.print(umidade, 1);
lcd.print(" %  ");

// Atualiza o Serial Monitor
Serial.print("[INFO] Temp: ");
Serial.print(temperatura, 1);
Serial.print(" C | Umid: ");
Serial.print(umidade, 1);
Serial.println(" %");
break;

```

case 2: // 3. DISTÂNCIA DO SENSOR E VELOCIDADE

```

// Atualiza o LCD
lcd.setCursor(0, 0);
lcd.print("Mts Sensor: ");
lcd.print((int)distanciaAnterior);
lcd.print(" ");
lcd.setCursor(0, 1);
lcd.print("Vel: ");
lcd.print((int)velocidadeDisplay);
lcd.print(" km/h  ");

```

```
// Atualiza o Serial Monitor
Serial.print("[INFO] Mts Sensor: ");
Serial.print((int)distanciaAnterior);
Serial.print(" m | Vel: ");
Serial.print((int)velocidadeDisplay);
Serial.println(" km/h");
break;
```

case 3: // 4. DISTÂNCIA PARA A PRÓXIMA CIDADE

```
// Atualiza o LCD
lcd.setCursor(0, 0);
if (metrosParaCidade > 0) {
    lcd.print("Faltam ");
    lcd.print((int)metrosParaCidade);
    lcd.print(" mts ");
    lcd.setCursor(0, 1);
    lcd.print("proxima cidade ");

    // Atualiza o Serial Monitor
    Serial.print("[INFO] Faltam ");
    Serial.print((int)metrosParaCidade);
    Serial.println(" mts para a proxima cidade");
} else {
    lcd.print("Voce chegou! ");
    lcd.setCursor(0, 1);
    lcd.print(" ");

    // Atualiza o Serial Monitor
    Serial.println("[INFO] Voce chegou ao seu destino!");
}
break;
```

case 4: // 5. MENSAGEM FINAL

```
// Atualiza o LCD
lcd.setCursor(2, 0);
lcd.print("BOA VIAGEM! ");
lcd.setCursor(2, 1);
lcd.print("Dirija bem. ");

// Atualiza o Serial Monitor
Serial.println("[INFO] BOA VIAGEM! Dirija bem.");
break;
}
}
```

Files

main

Go to file

ProjetoPMW

Display.ino

LDR.ino

ProjetoPMW1.ino

sensorDHT.ino

ultrassonico.ino

README.md

projeto-PMV / ProjetoPMW / ProjetoPMW1.ino

luizasilva08 Adicionando abas 1582eb · 1 hour ago History

Code Blame 71 lines (59 loc) · 1.56 KB

```
1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3 #include <DHT.h>
4
5 // --- PINOS ---
6 #define LDR_PIN A0
7 #define DHTPIN A1
8 #define TRIG_PIN 7
9 #define ECHO_PIN 8
10 #define LED_PIN 13
11
12 // --- CONFIGURAÇÕES DE OBJETOS ---
13 #define DHTTYPE DHT11
14 DHT dht(DHTPIN, DHTTYPE);
15 LiquidCrystal_I2C lcd(0x27, 16, 2);
16
17 // --- LIMITES ---
18 #define VELOCIDADE_MAX_DIA 10.0
19 #define VELOCIDADE_MAX_NOITE 20.0
20
21 // --- VARIÁVEIS GLOBAIS (compartilhadas entre as abas) ---
22 // Ultrassônico
23 unsigned long tempoAnterior = 0;
24 const long intervaloMedicao = 500;
25 float distanciaAnterior = 0;
26 float velocidadeAtual = 0;
27 float metrosPorSegundo = 5000.0;
28 float velocidadeDisplay = 0;
29 unsigned long temporicoVelocidade = 0;
30
31 // DHT11
32 unsigned long tempoAnteriorDHT = 0;
33 const long intervaloDHT = 2000;
34 float temperatura = 0;
```

Files

display

Go to file

ProjetoPMW

Display.ino

LDR.ino

ProjetoPMW1.ino

sensorDHT.ino

ultrassonico.ino

README.md

projeto-PMV / ProjetoPMW / Display.ino

EMILLY MIRANDA DREHMER display 9f13f69 · 3 minutes ago History

Code Blame 88 lines (79 loc) · 2.35 KB

```
1 void setup_Display() {
2   lcd.init();
3   lcd.backlight();
4   lcd.setCursor(0, 0);
5   lcd.print("Iniciando PMW...");
6 }
7
8 void atualizarRelogio() {
9   unsigned long tempoAtual = millis();
10  if (tempoAtual - tempoRelogioAnterior >= 1000) {
11    tempoRelogioAnterior = tempoAtual;
12    segundo++;
13    if (segundo >= 60) {
14      segundo = 0; minuto++;
15      if (minuto >= 60) {
16        minuto = 0; hora++;
17        if (hora >= 24) { hora = 0; dia++; }
18      }
19    }
20    if (telaAtual == 0) atualizarDisplay();
21  }
22 }
23
24 void loop_Display(unsigned long tempoAtual) {
25   atualizarRelogio();
26
27   // Temporizador para trocar a tela
28   if (tempoAtual - tempoTela >= intervaloTela) {
29     tempoTela = tempoAtual;
30     telaAtual = (telaAtual + 1) % 5;
31     limparTela = true;
32     atualizarDisplay();
33   }
34 }
```

luizasilva08 / projeto-PMV

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Files

sensor-dht

Go to file

ProjetoPMW

- Display.ino
- LDR.ino
- ProjetoPMW1.ino
- sensorDHT.ino**
- ultrassonico.ino
- README.md

projeto-PMV / ProjetoPMW / sensorDHT.ino

DOUGLAS DE SOUZA PAIXÃO Sensor DHT 3a774a0 · 44 minutes ago History

Code Blame 19 lines (16 loc) · 464 Bytes

```
1 void setup_DHT() {
2   dht.begin(); // Inicia o sensor DHT11
3 }
4
5 void loop_DHT(unsigned long tempoAtual) {
6   if (tempoAtual - tempoAnteriorDHT >= intervaloDHT) {
7     tempoAnteriorDHT = tempoAtual;
8     float t = dht.readTemperature();
9     float h = dht.readHumidity();
10
11     if (!isnan(t) && !isnan(h)) {
12       temperatura = t;
13       umidade = h;
14     }
15
16     // Atualiza se estiver na tela de Temp/umidade (Tela 1)
17     if (telaAtual == 1) atualizarDisplay();
18   }
19 }
```

Files

sensor-ldr

Go to file

ProjetoPMW

- Display.ino
- LDR.ino**
- ProjetoPMW1.ino
- sensorDHT.ino
- ultrassonico.ino
- README.md

projeto-PMV / ProjetoPMW / LDR.ino

amandade-miranda-maker Sensor LDR ca4e102 · 53 minutes ago History

Code Blame 15 lines (13 loc) · 473 Bytes

```
1 void setup_Luz() {
2   pinMode(LDR_PIN, INPUT);
3   pinMode(LED_PIN, OUTPUT);
4 }
5
6 void loop_Luz(unsigned long tempoAtual) {
7   // Define se é dia ou noite e atualiza o limite global
8   estaDeNoite = (analogRead(LDR_PIN) > 512);
9   limiteAtual = estaDeNoite ? VELOCIDADE_MAX_NOITE : VELOCIDADE_MAX_DIA;
10
11   // Temporizador de desligamento do LED de infração
12   if (digitalRead(LED_PIN) == HIGH && (tempoAtual - tempoUltimaInfraçao >= 3000)) {
13     digitalWrite(LED_PIN, LOW);
14   }
15 }
```

Files

sensor-distancia

+ Q

Go to file

ProjetoPMW

- Display.ino
- LDR.ino
- ProjetoPMW1.ino
- sensorDHT.ino
- ultrassonico.ino
- README.md

projeto-PMV / ProjetoPMW / ultrassonico.ino

lulzasilva08 Sensor ultrassônico a58ae40 · 26 minutes ago History

Code Blame 52 lines (42 loc) · 1.42 KB

```
1 void setup_Ultra() {
2   pinMode(TRIG_PIN, OUTPUT);
3   pinMode(EG0_PIN, INPUT);
4 }
5
6 float lerDistancia() {
7   digitalWrite(TRIG_PIN, LOW);
8   delayMicroseconds(2);
9   digitalWrite(TRIG_PIN, HIGH);
10  delayMicroseconds(10);
11  digitalWrite(TRIG_PIN, LOW);
12
13  long duracao = pulseIn(EG0_PIN, HIGH, 30000);
14  if (duracao == 0) return distanciaAnterior;
15  return (duracao * 0.0343) / 2;
16 }
17
18 void loop_Ultra(unsigned long tempoAtual) {
19   if (tempoAtual - tempoAnterior > intervaloMedicao) {
20     float distanciaAtual = lerDistancia();
21     float deltaDistancia = abs(distanciaAnterior - distanciaAtual);
22     float deltaTempo = intervaloMedicao / 1000.0;
23
24     velocidadeAtual = (deltaDistancia / deltaTempo) * 3.6;
25
26     if (deltaDistancia > 0) {
27       metrosParaCidade = metrosParaCidade - deltaDistancia;
28       if (metrosParaCidade < 0) metrosParaCidade = 0;
29     }
30
31     if (velocidadeAtual > 5.0) {
32       velocidadeDisplay = velocidadeAtual;
33       tempoSicovelocidade = tempoAtual;
34     }
35   }
36 }
```